ES 666: Computer Vision

Assignment 2 — Find Waldo

The Spotter's Problem

You are volunteering at the lost-and-found desk of a very large outdoor festival. Someone walks up, worried, and hands you a single clear photograph of the friend they came with: a striped shirt, a bobble hat, round glasses. Behind you, covering the whole wall, is the panorama the drone took over the main field ten minutes ago. Two hundred people in it. Several of them are wearing stripes.

Your first instinct is the obvious one. You have a picture of the friend and you have a picture of the crowd, so slide the small picture over the big one and look for the place where the pixels agree best. This works, and it is worth understanding exactly why it works before you find out where it stops working.

Because it does stop. The friend in the panorama is not at the same size as in the photograph, the light on the field is not the light in the photograph, and at some point they turn their head or lean sideways and the whole comparison falls apart. When that happens you need a different idea: forget whole-image agreement, find a handful of small distinctive spots, describe each one in a way that survives being rotated and resized, match them, and let the matches vote on where the answer is.

Both ideas are in this assignment. Neither of them is a neural network, and neither of them is a library call. You will build both from `numpy` and compare them honestly.

You are given a small template of Waldo and four crowded scenes. Your job is to find him twice, using two completely different ideas, and then to say which one worked, where it failed, and why.

- **Tasks 1–2** build a *correlation* detector. It slides the template over the scene and asks "how well does the template match here?" at every position and every size.
- **Tasks 3–4** build a *feature* detector. It finds a few dozen distinctive points in the template and in the scene, describes each one, matches them, and lets the matches vote on where Waldo is.

The two behave very differently, and the last part of the assignment is about that difference. Neither one solves every scene — that is the point, not a bug in the assignment.

Ground rules

1. **NumPy only for the algorithms.** No `cv2.filter2D`, `cv2.matchTemplate`, `cv2.SIFT`, `cv2.goodFeaturesToTrack`, `scipy.ndimage`, `skimage`, or anything similar. The provided helpers use `cv2` for reading and resizing images only, and that is fine — resampling is not what this assignment teaches.
2. **No deep learning.** No PyTorch, no TensorFlow, no pretrained anything.

3. **Do not touch the helper functions or the imports.** They are marked in the files. Fill in only the functions whose body reads `return ... # comment this line and write your code for the function`.

Task 1 — Filtering, and knowing whether it helped

The scenes are noisy. Before you can match anything you need to clean them up, and before you can claim you cleaned them up you need a number that says so.

Functions to implement

1.1 `convolve2d(image, kernel)`
True convolution: flip the kernel on both axes, pad by half the kernel with `mode="reflect"`, and slide. Output is the same shape as the input. Loop over the *kernel*, not over the pixels — a 9×9 kernel is 81 whole-array operations, while looping pixels is hundreds of thousands of Python iterations.

The input may be `(H, W)` or `(H, W, 3)` and one code path must handle both. Pad only the two *spatial* axes and the arithmetic broadcasts over colour by itself; padding the channel axis as well is the most common bug in this function.

1.2 `gaussian_kernel(size, sigma)`
Sample the Gaussian on a centred grid and normalise by the *sum of the samples*, not by the analytic constant $1/(2\pi\sigma^2)$. Be ready to say why those two differ on a finite grid.

1.3 `median_filter(image, size)`
The median of each window. Not a convolution — no kernel exists that computes a median. On colour, take each channel's median separately; question 1.10 is about what that costs you.

1.4 `sobel_edges(image)`
Returns `gx`, `gy`, `magnitude`, `orientation`. Orientation in degrees in $[0, 360)$. All four outputs are 2-D even when the input is RGB, because the gradient is taken on `to_grayscale(image)`.

1.5 `psnr(reference, test)`
PSNR $= 10 \log_{10}(1/\text{MSE})$ for images in $[0, 1]$. Identical images give ∞.

1.6 `ssim(reference, test)`
Structural similarity, using a Gaussian window for the local means, variances and covariance, with $C_1 = 0.01^2$ and $C_2 = 0.03^2$. Return the mean score and the full map.

Written

1.7 Predict first, then measure.
Before you run anything, write down two predictions: will a 9×9 box filter make PSNR better or worse on the Gaussian-noised image, and will it make SSIM better or worse? Commit to both in writing.

Now run `task1.py` and report the four numbers from the first row of your table to two decimal places. State plainly whether each prediction was right. A wrong prediction honestly reported and then explained earns full marks; a prediction that suspiciously matches the measurement earns none.

1.8 The two metrics disagree.
One of those two numbers goes up while the other goes down. Explain how one image can be simultaneously further from the original and more similar to it. Which would you trust if the next step is template matching, and why?

1.9 What colour costs you.
Re-run the whole table on `to_grayscale(clean)` so every filter and both metrics see a single channel. Paste both tables and the lines you changed. PSNR and SSIM barely move — explain why that is expected, and why it does *not* mean the colour is useless. (Task 2 is where the answer shows up.)

1.10 Padding.
Switch `convolve2d` to zero padding, blur with a large kernel and look at the border. Describe the artefact and say why reflect is the safer default before a template match.

Task 2 — Finding Waldo by sliding the template

The direct approach. Put the template at every position, score the match, and take the best. Doing that naively is far too slow, so most of this task is about making it fast enough to actually run.

Functions to implement

2.1 `integral_image(image)`
A summed-area table with a zero first row and column. Two cumulative sums.

2.2 `box_sum(ii, y0, x0, h, w)`
The sum over any box in four lookups, regardless of box size. `y0` and `x0` are arrays, so this does all windows at once.

2.3 `ncc_map(image, template)`
Zero-mean normalised cross-correlation at every position:

$$\mathrm{NCC} = \frac{\sum (I - \bar{I})(T - \bar{T})}{\sqrt{\sum \|I - \bar{I}\|^2}\,\sqrt{\sum \|T - \bar{T}\|^2}}$$

2.4 `find_peaks(score_map, threshold, min_distance, max_peaks)`
Strongest first, keeping a peak only if it is at least `min_distance` from every peak already kept, in Chebyshev distance.

2.5 `non_max_suppression(boxes, scores, iou_threshold)`
Standard greedy NMS. A box exactly *at* the threshold survives.

2.6 `detect_multiscale(image, template, scales)`
Score at every scale, pool all the peaks, then suppress across scales together — not scale by scale.

Written

2.7 Why NCC and not sum-of-squared-differences?
Show algebraically that NCC is unchanged by $I \to aI + b$ for $a > 0$, and say what SSD does under the same change.

2.8 Grayscale against colour, measured.
Re-run the task with `to_grayscale` applied to both the scene and the template. Report the two tables side by side: which scenes still work, and how much the winning score moves. Relate the size of that change to Waldo's actual colours. Paste the lines you changed.

2.9 The mask you were given and never used.
`imgs/waldo_mask.png` ships with this assignment and no task touches it. It marks which pixels inside the template rectangle are actually Waldo rather than background, which matters because `scene_hard` has an occluder over part of him.

Write a masked version of `ncc_map` that correlates only over pixels where the mask is set (centre each channel by its masked mean, and normalise by the masked norms). Report the best score and its scale on `scene_hard`, with and without the mask.

Do not predict the answer from theory — measure it, and report what you actually got even if it surprises you. Then explain the direction of the change in terms of what the unmasked correlation is forced to include.

2.10 Scores are not comparable across scales.
A smaller template has fewer pixels, so its NCC has a wider spread and its maximum over a whole scene is higher *by chance alone*. For `scene_hard`, print `.max()` and `.std()` of the score map at every scale, alongside the template's pixel count, and paste the table.

Look at the spread across scales before you answer this: would dividing each map by its own standard deviation make the scales comparable? Justify your answer from your own numbers, not from what standardisation usually does.

Task 3 — Corners, blobs and descriptors

A different idea. Instead of comparing whole rectangles, find a few dozen *distinctive points*, and describe the neighbourhood of each one in a way that survives the image being resized or rotated.

Functions to implement

3.1 `harris_response(image, k, sigma)`
Build the structure tensor from the Sobel gradients you wrote in Task 1, smooth its three entries with a Gaussian, and return $\det(J) - k\,\mathrm{trace}(J)^2$.

3.2 `detect_corners(response, threshold_ratio, min_distance)`
Threshold relative to the maximum response, then keep the strongest well-separated ones.

3.3 `dog_keypoints(image, sigma0, n_scales, ratio, ...)`
Build a Gaussian ladder, difference adjacent levels, and keep points that are extremal among their 26 neighbours in the $3 \times 3 \times 3$ space–scale cube. Returns (y, x, σ): unlike a Harris corner, a blob has a *size*, and that size is what makes Task 4 work.

3.4 `dominant_orientation(image, y, x, sigma)`
A gradient-orientation histogram over a Gaussian-weighted patch, scaled to σ. Return the peak bin, in degrees.

3.5 `describe_keypoints(image, keypoints, orientations)`
A 4×4 grid of 8-bin orientation histograms, sampled in a frame rotated by the keypoint's orientation and scaled by its σ — 128 numbers per keypoint. Normalise each descriptor to unit length, except that an all-zero descriptor stays all-zero.

Written

3.6 Corner or blob?
A Harris corner is (y, x); a DoG keypoint is (y, x, σ). Explain where that third number comes from, and why Task 4 could not work with Harris corners alone.

3.7 The ladder is a trade.
Run `dog_keypoints` on the template with at least three different `(sigma0, n_scales, k)` settings. Report the keypoint count and the σ range for each, and say what breaks at both ends of the ladder.

3.8 More keypoints, fewer matches.
`find_waldo` enlarges the template by `upsample=2.0` before detecting. Bigger ought to be better: more pixels, more keypoints, more chances to match.

Test it. Run `find_waldo` on `scene_easy` with `upsample` set to 1.5, 2.0, 3.0 and 4.0, and for each one report two numbers: how many keypoints were found on the template, and how many matches survived (`n_matches`). Paste the loop you wrote.

The trend in the second column is not the one you would guess. State what it is, then explain it. Your explanation must refer to a specific visual property of Waldo's shirt and to the exact condition inside `match_ratio_test` that this property triggers. An answer that only says "more keypoints means more noise" has not understood it.

3.9 Why does this task throw the colour away?
Tasks 1 and 2 work in RGB; this one reduces to luminance. Explain what goes wrong if you try to define a gradient *orientation* on a three-channel image directly, using a red-to-green edge as your example.

Task 4 — Matching, and letting the matches vote

You have descriptors on the template and on the scene. Now match them — and deal with the fact that most of the matches will be wrong.

Functions to implement

4.1 `descriptor_distances(desc_a, desc_b)`
All-pairs Euclidean distance from one matrix product, using $\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2a{\cdot}b$. No double loop. Clip at zero before the square root.

4.2 `match_ratio_test(desc_scene, desc_template, ratio)`
Lowe's ratio test — but read the docstring before you write it, because the DIRECTION is the whole point and it is the opposite of the obvious one. Each *scene* keypoint asks which of the ∼80 template parts it resembles, not the other way round, and there is deliberately **no** mutual-nearest-neighbour check.

4.3 `predict_centres(...)`
The heart of the task. A match pairs two keypoints that each carry a position, a scale and an orientation — so a *single* match determines the size change $s = \sigma_s/\sigma_t$, the rotation $\Delta\theta = \theta_s - \theta_t$, and therefore a full guess at where the centre of Waldo is:

$$
\begin{aligned}
(o_x, o_y) &= (W_t/2 - x_t,\ H_t/2 - y_t) \\
c_x &= x_s + s\,(\cos\Delta\theta \cdot o_x - \sin\Delta\theta \cdot o_y) \\
c_y &= y_s + s\,(\sin\Delta\theta \cdot o_x + \cos\Delta\theta \cdot o_y)
\end{aligned}
$$

Vectorise over all matches. A wrong match will place its guess anywhere at all, possibly outside the image. That is expected.

4.4 `consensus_centre(predictions, tol, min_votes)`
Find the *densest cluster* of guesses: count how many guesses lie within `tol` of each guess, take the winner as a seed, and keep everything within `tol` of it. Report the median of that cluster, or `None` if it has fewer than `min_votes` members.

4.5 `find_waldo(scene, template, upsample)`
Wire it together and return a box, or `None` when there is no consensus. Do not invent a box.

Written

4.6 Why the median fails.
Only a small fraction of the matches are correct. Replace the densest-cluster step with a plain median over *all* the predictions, report the IoU you get on each scene, and explain the result in terms of what fraction of the data has to be correct for a median to mean anything.

4.7 The abnormality: each method fails where the other does not.

(a) Name the exact thing missing from Task 2's search space, and prove your claim by explaining why adding twenty more entries to `SCALES` cannot help.

(b) Point to the specific step that buys Task4, and say what would happen on `scene_rotated` if you passed `orientations=None` to `describe_keypoints`.

(c) Explain why the more capable method is the less precise one. Mention what Task 2 returns exactly and Task 4 only estimates.

4.8 The check that every tutorial recommends.
`match_ratio_test` deliberately omits a mutual-nearest-neighbour test: it does not require that a scene keypoint's best template partner also picks that scene keypoint back. Almost every published description of feature matching includes one.

Add it — keep a match only when `np.argmin(distances, axis=0)[j1] == rows` — and re-run all four scenes. Report `n_matches` and `n_votes` for each, with and without. Paste the line you added.

Report honestly what happened, then explain it. Your answer must say something about how many scene keypoints can legitimately correspond to the *same* template part, and why that makes a one-to-one pairing the wrong model for finding an object in a scene.

4.9 Agreement without an answer key.
The last column is the IoU between your Task 2 box and your Task 4 box — no ground truth involved. Explain why disagreement is informative even though it cannot tell you *which* detector is wrong, and construct a concrete case where two independent detectors would agree and both be wrong.

What to submit

A single archive named `<rollnumber>_assignment2.zip` containing:

- `task1.py`, `task2.py`, `task3.py`, `task4.py`

- `subjective_answers_task1.md` ... `subjective_answers_task4.md`
- your `plots/` folder

Do not include the virtual environment or the `imgs/` folder.